Joy Node

Thumrongsak Kosiyatrakul

Joy Node

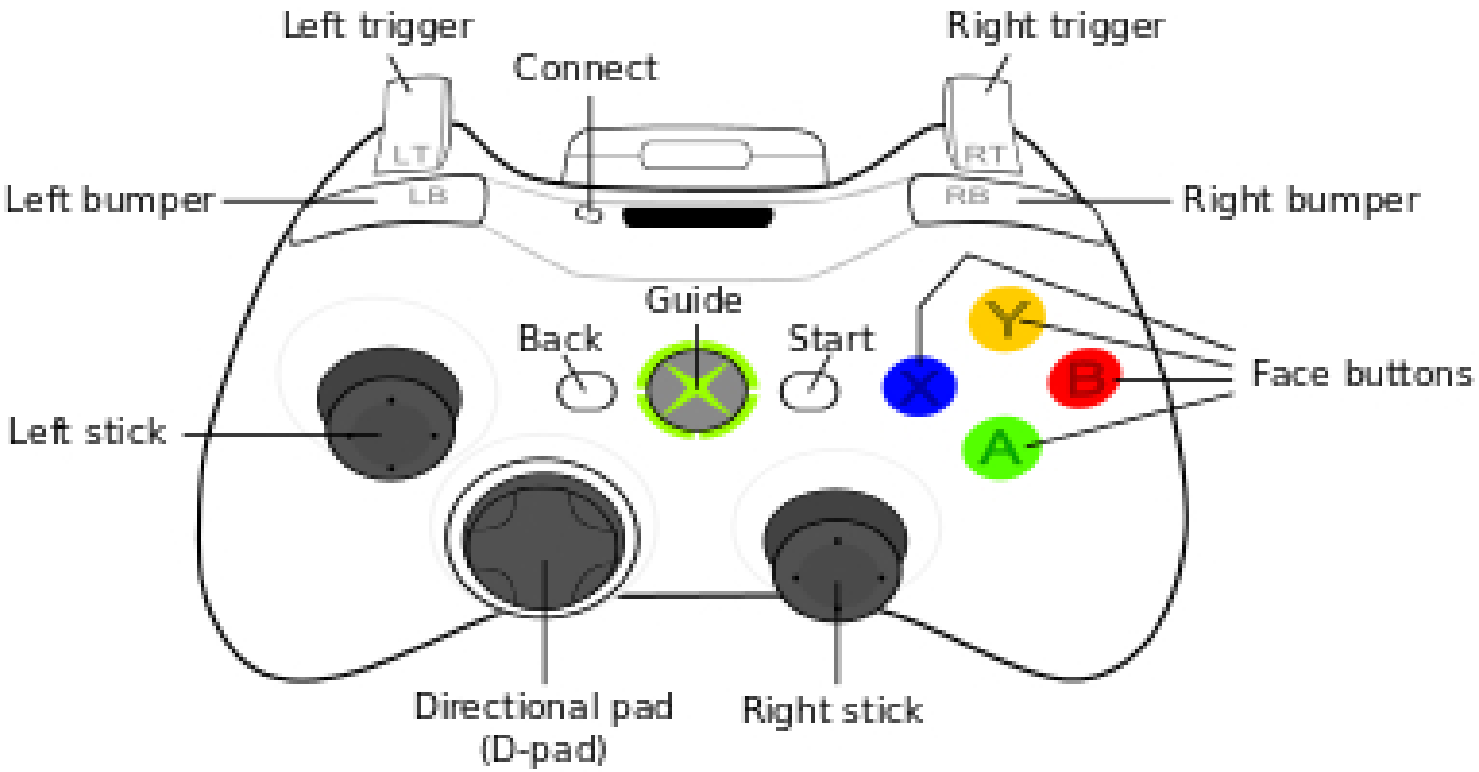
- `joy_node` node communicates with a joystick connected via a USB port
- Provide hardware abstractions
- To receive data from a joystick, simply subscribe for messages from the topic named `joy`
- To start a joy node, use the following series of commands on a console screen:

```
rosparam set joy_node/dev "/dev/input/js1"
rosrun joy joy_node
```

- Make sure `roscore` has been executed in another console screen
- The command `rostopic list` should show the following:

```
/diagnostics
/joy
/rosout
/rosout_agg
```

Xbox 360 Controller



Topic `joy`

- The `joy` node publishes joystick data to a topic named `joy`
- The type of message is `Joy`
- The message of type `Joy` is a data structure consisting of the following components:
  1. Data structure named `header`,
  2. Array of floating-point numbers named `axes`, and
  3. Array of integers named `buttons`

- Array of integers named `buttons` are organized as follows:

Index	Button name on the controller
0	A
1	B
2	X
3	Y
4	LB
5	RB
6	back
7	start
8	power
9	Button stick left
10	Button stick right

- Array of floating-point numbers named `axes` are organized as follows:

Index	Button name on the controller
0	Left/Right Axis stick left
1	Up/Down Axis stick left
2	LT
3	Left/Right Axis stick right
4	Up/Down Axis stick right
5	RT
6	cross key left/right
7	cross key up/down

Topic joy

- **Note** that if the state of joystick does not change, no messages will be published to the topic joy
- If you run the command

```
rostopic echo -c joy
```

  you may not see anything
- If you press the button A, you may see something as shown below:

```
header:
  seq: 265
  stamp:
    secs: 1464206519
    nsecs: 129026422
  frame_id: ``
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, 0.0, 0.0]
buttons: [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Topic `joy`

- To import the data structure `joy` to your Python script, you need to insert the following statement in the beginning:

```
#!/usr/bin/env python

import rospy
from sensor_msgs.msg import Joy
```

- To subscribe to the topic `joy` of type `Joy`, use the following statement in your Python script:

```
    rospy.Subscriber("joy", Joy, callback)
```

Example

```
#!/usr/bin/env python

import rospy
from sensor_msgs.msg import Joy

def callback(data):
    print data.buttons[0]
    print data.axes[0]

def testJoy():
    rospy.init_node("joystick", anonymous=True)
    rospy.Subscriber("joy", Joy, callback)
    rospy.spin()

if __name__ == '__main__':
    testJoy()
```

Controlling a Kobuki

- A robot can be controlled by sending a message of type `Twist`
- To import the data structure `Twist`, use the following command:

```
from geometry_msgs.msg import Twist
```

- To create a variable named `command` of type `Twist`, use the following command:

```
command = Twist()
```

- We will only use two components:
  - `command.linear.x`
    - Value can be between -1.0 to 1.0
    - Positive value makes the robot moves forward
    - Negative value makes the robot moves backward
  - `command.angular.z`
    - Value can be between -1.0 to 1.0
    - Positive value makes the robot turns to its left
    - Negative value makes the robot turns to its right

Controlling a Kobuki

- To control the robot, the message of type `Twist` must be publish to the following topic:

```
/mobile_base/commands/velocity
```

- Examples:
  - If you want to make the robot moves forward in a straight line at half speed:

```
command.linear.x = 0.5
command.angular.z = 0.0
```

  - For example, if you want to make the robot makes a stationary turn to its left at half speed:

```
command.linear.x = 0.0
command.angular.z = 0.5
```

  - For example, if you want to make the robot moves forward and turn at the same time:

```
command.linear.x = 0.5
command.angular.z = 0.5
```

Use Simulator (for now)

- ROS allows you to see how robot moves in a virtual world
- First, run the following command in a console screen:

```
roslaunch kobuki_softnode full.launch
```

- Open a new console screen and run the following command:

```
rosrun rviz rviz
```

- In `rviz`
  1. On the left pane, set Fixed Frame to `/odom`
  2. At the bottom left corner, click Add and choose `RobotModel`

  You should see an image of a Kobuki robot on your screen
- At this point, if you publish a command of type `Twist`, robot will move according to the command